ARTIFICIAL INTELLIGENCE · LECTURE 07+08

Playing Against an Opponent

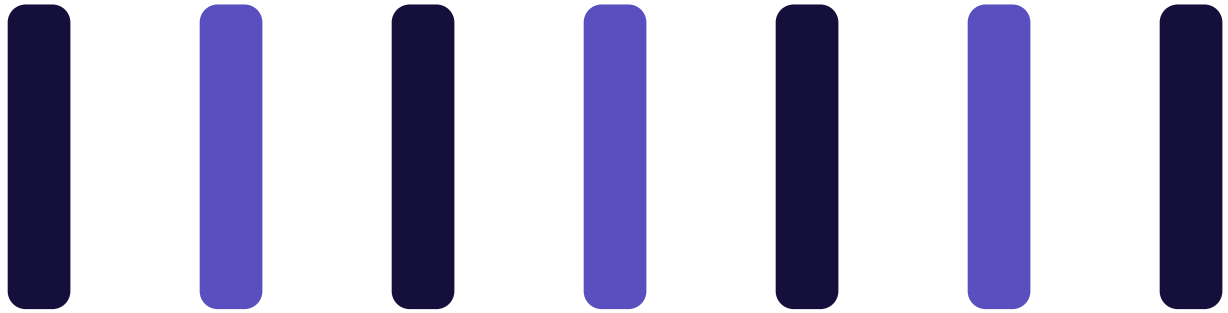
Minimax & Alpha-Beta Pruning: think like a champion



WARM-UP · 2 MINUTES · PLAY WITH YOUR NEIGHBOUR

Can You Beat Your Neighbour?

The Game of Sticks. Play it three times before reading on.



7 sticks on the table

THE RULES

1. Players take turns.
2. On your turn, take 1 or 2 sticks.
3. **Whoever takes the LAST stick wins.**

AFTER YOU PLAYED, ANSWER

- Is it better to go FIRST or SECOND?
- Is there a move that always wins? How did you decide what to take?
- Could a computer decide this without “feeling” anything? Yes, with Minimax. We will solve a smaller version in the worked example.

REAL LIFE

You Already Think Like Minimax

Whenever the other side reacts to your choice, you plan for their best reply.

Cricket captain

You choose a bowler. The batting captain will pick the batter who hits him best. A smart captain asks: “What is the worst that can happen with this bowler?”

Price war

Two mobile shops on the same road. If you drop your price, the other shop may drop theirs. You plan for their smartest reaction before you change your board.

Chess and carrom

Every strong player thinks: “If I play this, he plays that, then I play…”. A computer does the same, but without getting tired.

Security planning

A bank defends its system assuming the attacker is skilled, not careless. Planning for the opponent’s best move is the same worst-case thinking.

OUR RUNNING EXAMPLE

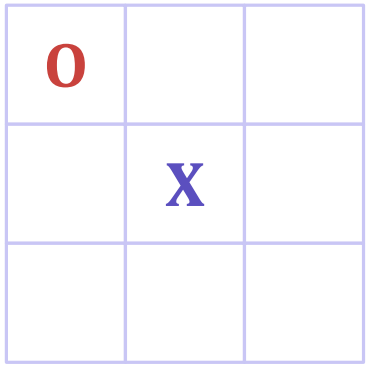
Tic-Tac-Toe: A Game as a Tree

MAX plays X. MIN plays O. Each move creates a new board, a new node.

START

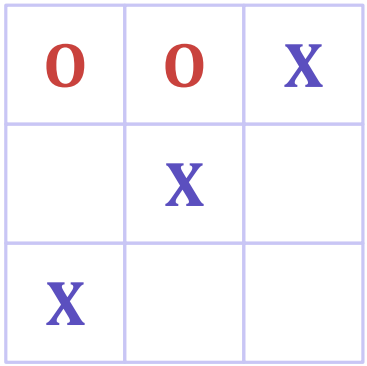
X to move

AFTER A FEW MOVES



the tree grows

END (TERMINAL)



X wins → +1

UTILITY (THE SCORE)

X wins +1

Draw 0

O wins −1

MAX wants the biggest number.
MIN wants the smallest.

Fun fact: there are 255,168 different tic-tac-toe games. Minimax can search all of them and proves that if both players play perfectly the result is always a DRAW (value 0). So going first does not guarantee a win!

AIMA SECTION 5.2

The Minimax Idea

Assume your opponent always plays their very best, then pick your best move.

MAX **picks the HIGHEST value**

MAX is us: we want the best score we can get.

Example: choices worth 3, 9, 6 → MAX takes 9.

MIN **picks the LOWEST value**

MIN is the opponent: they want us to score as low as possible.

Example: choices worth 3, 9, 6 → MIN takes 3.

REAL LIFE

Before moving your queen you ask: “If I do this, what is my opponent’s best reply? And my best answer after that?” You trust that the opponent will NOT make a silly mistake. That cautious, worst-case thinking is exactly Minimax.

The score of a position = the best score MAX can force, assuming MIN also plays perfectly.

When Does Minimax Fit?

The classic model assumes a predictable game with two players who can see the full position.

1

Two players

MAX tries to raise the score; MIN tries to lower it.

2

Taking turns

One player moves, then the other responds.

3

Deterministic

No dice or random events; a move has a predictable result.

4

Fully visible

Both players can see the complete board and position.

5

Zero-sum

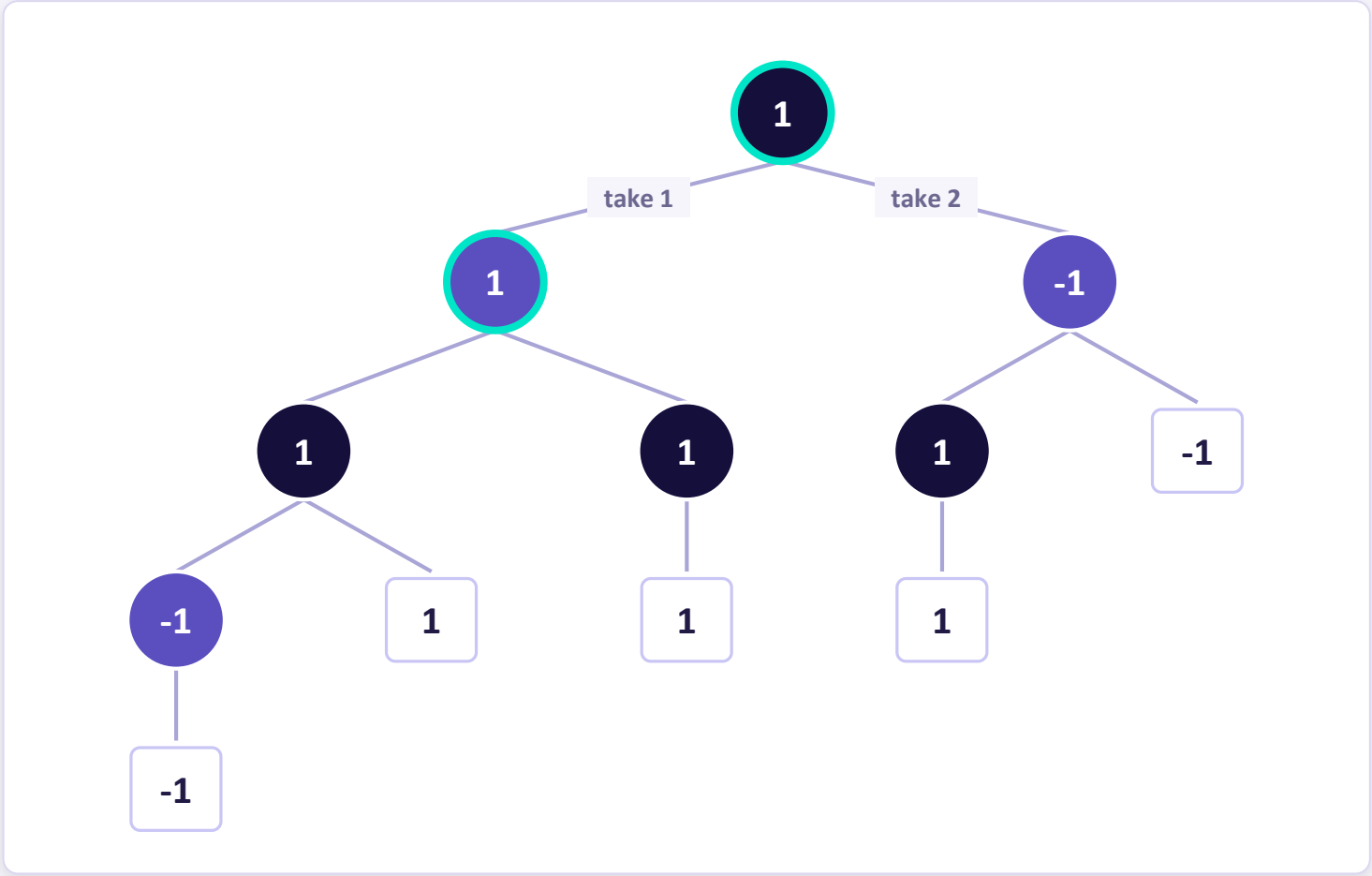
One player's gain is the other's loss; a draw scores 0.

GOOD EXAMPLE

Tic-Tac-Toe fits all five assumptions. A dice game needs chance nodes, so plain Minimax is not enough.

Sticks Solved: MAX Takes 1 and Wins

Worked example: start with 4 sticks. Each player may take 1 or 2.



RESULT

Root = +1. MAX takes 1 stick, leaving 3.

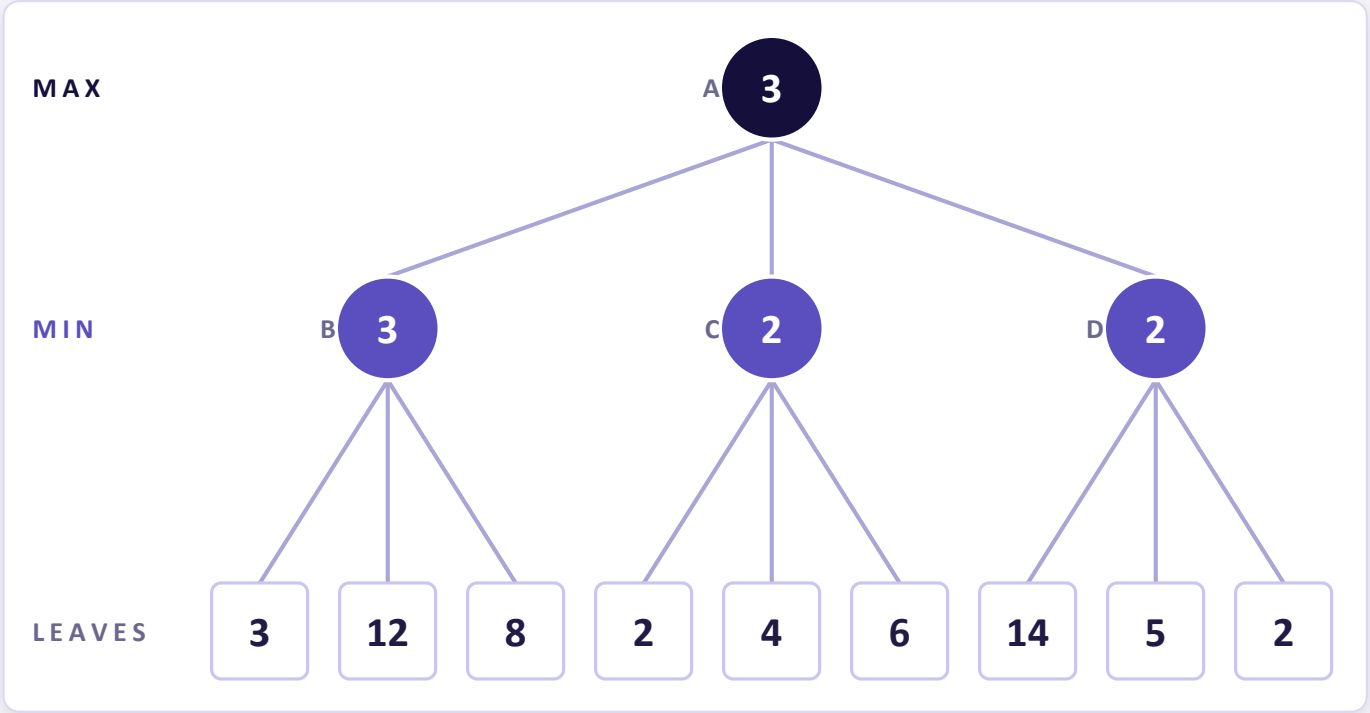
Whatever MIN does next, MAX can still take the last stick. Taking 2 instead (leaving 2) lets MIN win.

THE SECRET PATTERN

Always leave your opponent a multiple of 3 (3, 6, 9…). In the 7-stick warm-up, take 1 and leave 6. This worked tree starts with 4; MAX takes 1 and leaves 3.

Minimax Trace: Work From the Bottom Up

MIN nodes take the smallest child. Then MAX takes the biggest of those.



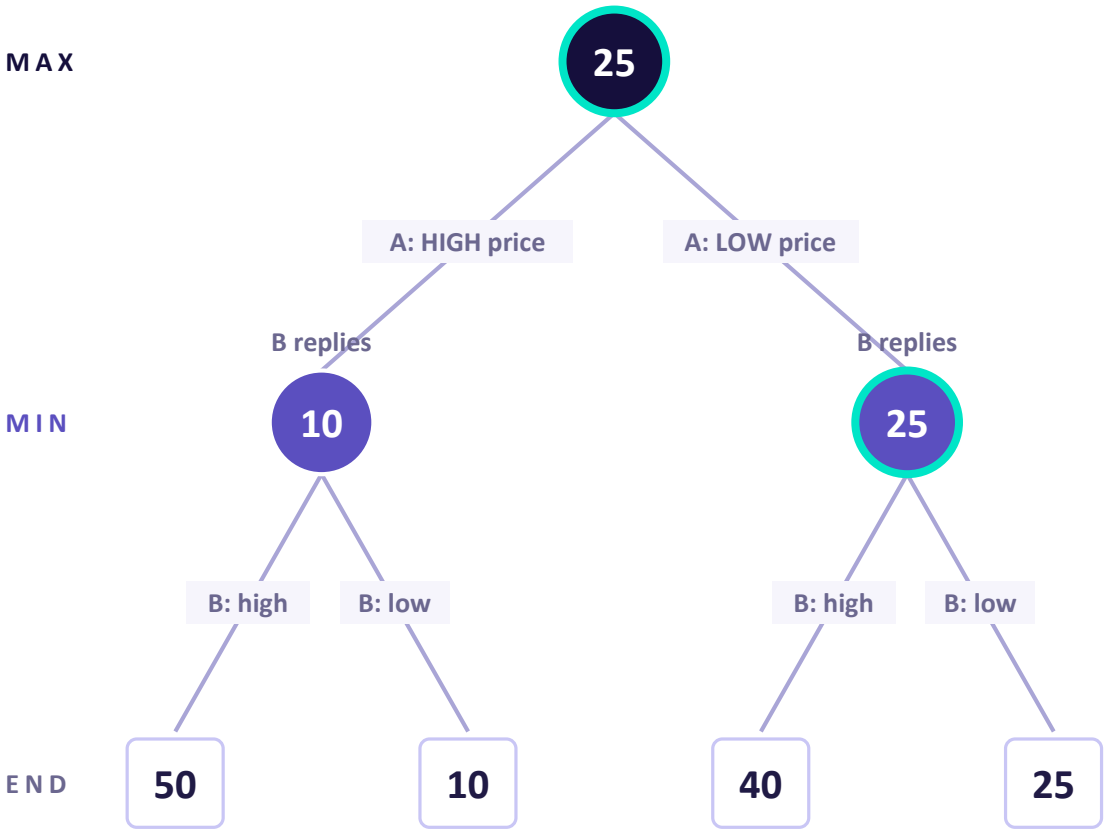
Node	Children	Rule	Value
B (MIN)	3, 12, 8	smallest	3
C (MIN)	2, 4, 6	smallest	2
D (MIN)	14, 5, 2	smallest	2
A (MAX)	**3, 2, 2**	**biggest**	**3**

Best move for MAX: B. It guarantees 3, even if MIN replies perfectly.

Why not D with its 14? Because MIN would never allow it. After D, MIN replies with 2. MAX must expect the opponent's best reply, not hope for a mistake.

The Price War

Shop A sets a price first. Shop B sees it and replies. The numbers are Shop A's monthly profit in Rs thousand.



THINK LIKE MINIMAX

If A sets a HIGH price, a smart B undercuts and A earns only **10**.

If A sets a LOW price, even B's best reply leaves A with **25**.

DECISION

A picks the LOW price: the best of the worst cases (min = 10 vs min = 25).

Notice: A never hopes that B makes a mistake.

Tic-Tac-Toe: Which Move Should X Play?

X (MAX) is to move. Three squares are free. Minimax checks every reply by O.

CURRENT BOARD

X	X	O
O	X	
		O

X to move

X	X	O
O	X	
	X	O

Value +1

X completes the middle column and wins at once.

X	X	O
O	X	X
		O

Value 0

X blocks O's right column. Then O must block X's middle column. The game ends in a draw.

X	X	O
O	X	
X		O

Value −1

X ignores O's threat. O plays the right-middle square and wins.

Minimax rule: X picks the move with the biggest value: +1. For every candidate, the value already assumes O replies as well as possible. A computer finds this by checking all the replies, exactly like the sticks tree.

THE PROBLEM

Real Games Are Far, Far Too Big

Minimax looks at EVERY node. Fine for sticks and tic-tac-toe, hopeless for chess.

GAME	SIZE OF THE GAME TREE	CAN MINIMAX FINISH?
Sticks (4)	12 nodes	Instantly
Tic-tac-toe	255,168 possible games	In a blink
Chess	about 10^{154} nodes (35^{100})	Not feasible
Go	even larger than chess	Not feasible

Atoms in the universe ≈ 10^{80}

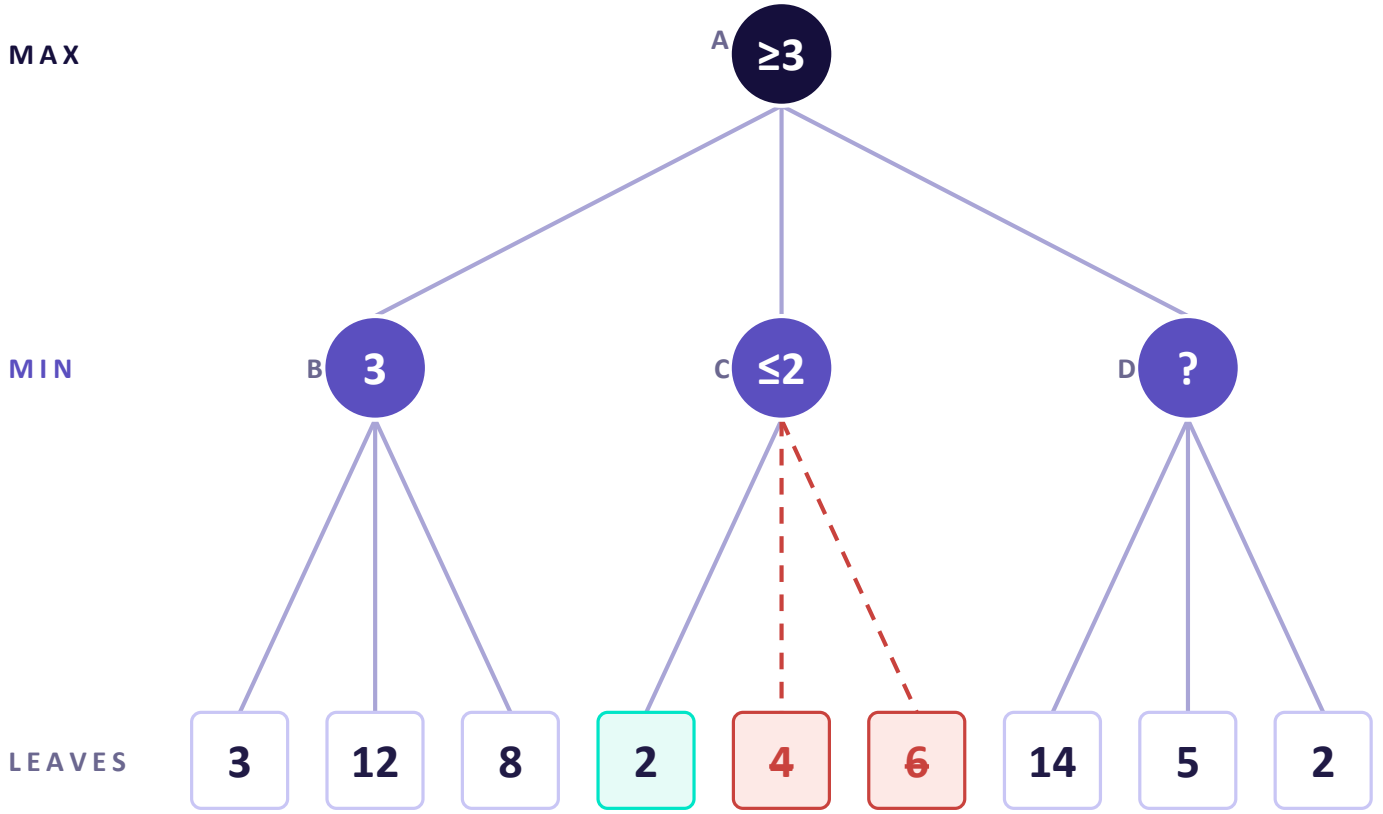
The chess tree is vastly bigger than even that number. The full tree is far beyond exhaustive search with available computing resources.

Good news: many branches provably cannot change the final answer. We can skip them and lose nothing.

THE KEY OBSERVATION

Do We Really Need to Look at Every Leaf?

Watch what happens at C once its first leaf (2) is seen.



THE LOGIC

1. B gave MAX a sure 3. So A is at least 3.
2. At C, the first leaf is 2. MIN can already hold MAX to 2 or less.
3. MAX already has 3 elsewhere, so C is useless. Whatever 4 and 6 are, MIN can pick 2.

Leaves 4 and 6 never need to be looked at.

Alpha-Beta Pruning: The Idea

Carry two numbers while searching. When they cross, stop looking at that branch.

α (alpha) *Starts at −∞*

The best (highest) score MAX can already guarantee on the path so far.

MAX's safety net: "I can get at least this."

β (beta) *Starts at +∞*

The best (lowest) score MIN can already guarantee on the path so far.

MIN's safety net: "I can hold you to at most this."

THE PRUNING RULE

Prune as soon as $\alpha \geq \beta$

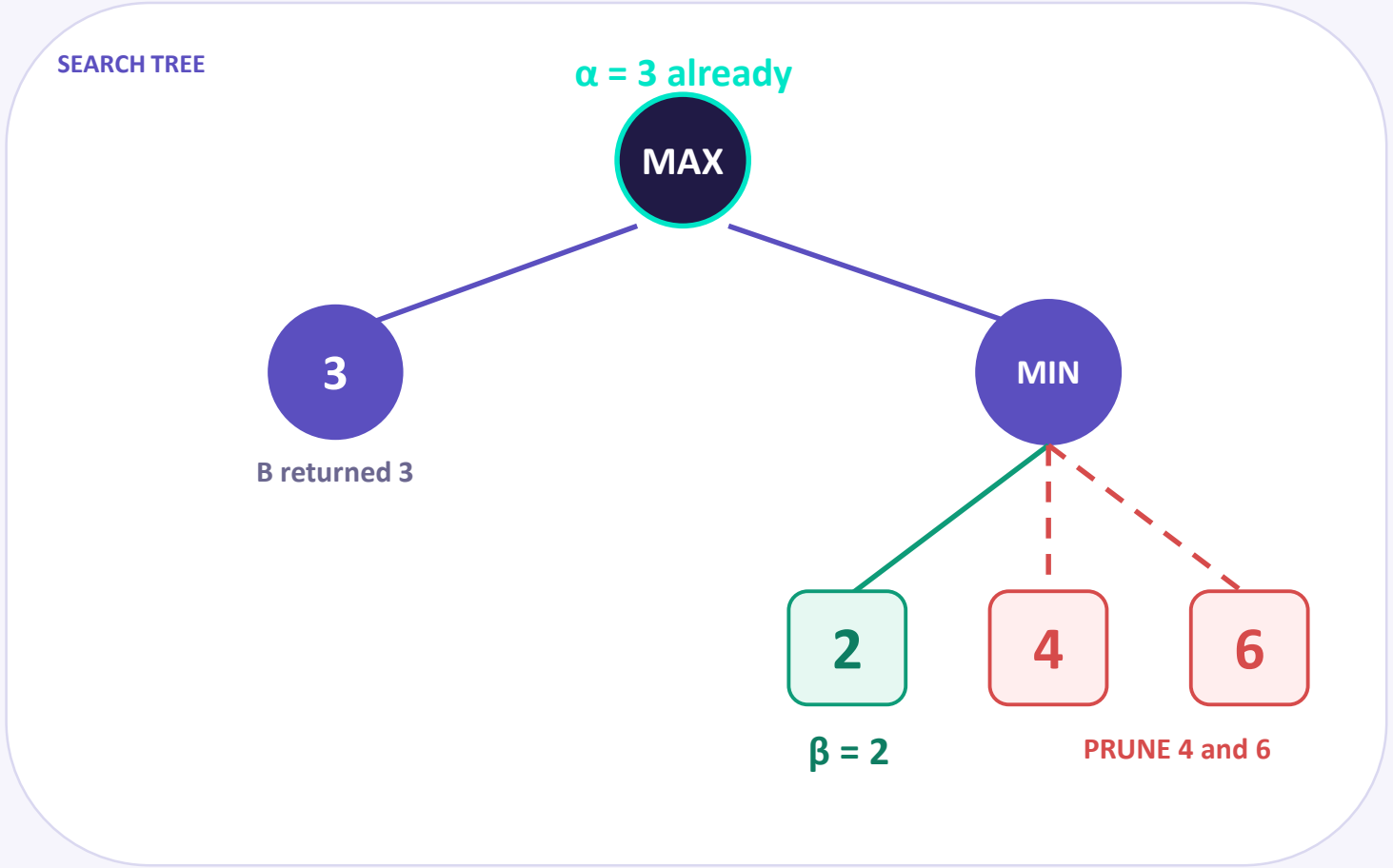
Skip the remaining children. Nothing there can change the answer.

SAY IT IN ONE LINE

"Stop exploring this branch when you already know your opponent will never let the game go here."

Why Can We Skip These Two Leaves?

MAX already has a guaranteed score of 3. At C, MIN can already hold MAX to 2.



1 **Earlier branch B gave MAX 3.**

So α = 3.

2 **At C, the first leaf is 2.**

So β = min(+∞, 2) = 2.

3 **Compare the bounds.**

β ≤ α → 2 ≤ 3, so stop.

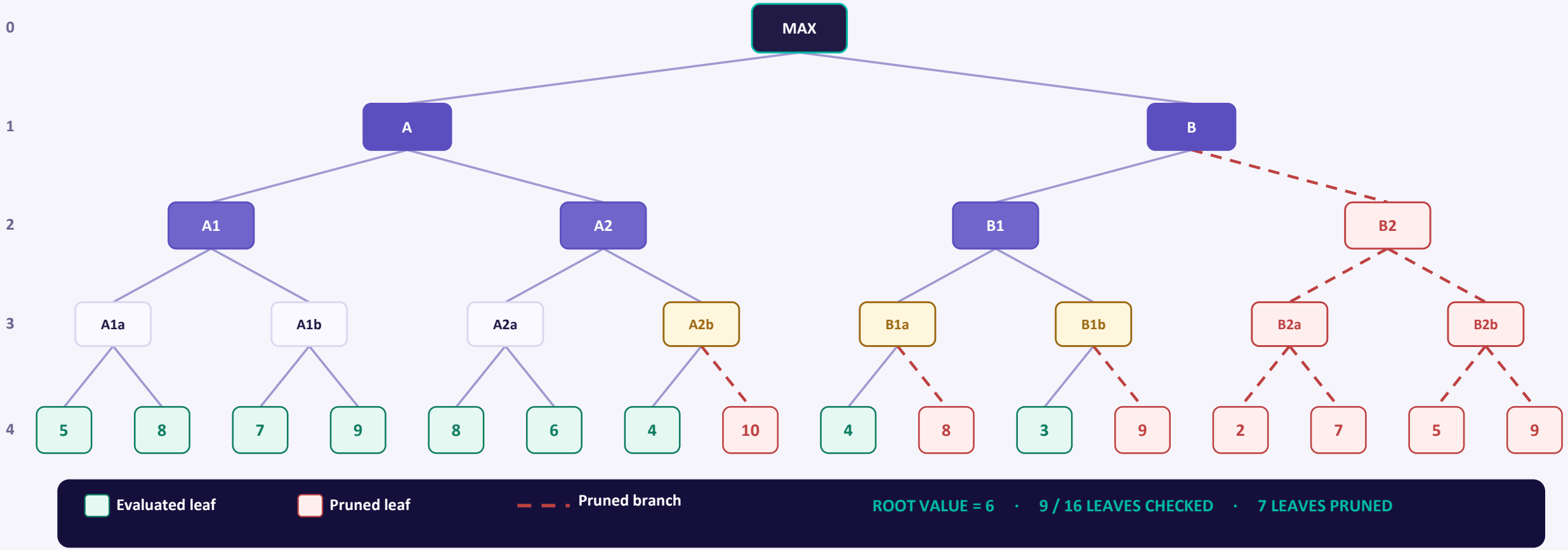
The answer stays 3; only extra checks are skipped.

Full Alpha-Beta Tree: Root to Depth 4

Depth 0 is the root; depth 4 contains 16 terminal scores. Search proceeds left to right.

α (MAX): best value so far | β (MIN): best value so far

MAX (α) → MIN (β) → MAX (α) → MIN (β) → SCORE



Alpha-Beta Trace: Follow Every Cutoff

Read the tree from left to right. Update α at MAX nodes and β at MIN nodes.

#	NODE	LEAVES VISITED	BOUND / UPDATE	ACTION
1	**A1a · MIN**	5, 8 → 5	No cutoff	**Return 5**
2	**A1b · MIN**	7, 9 → 7	α = 5; β = +∞	**Return 7**
3	**A1 · MAX**	max(5, 7) = 7	A updates β = 7	**Return 7 to A**
4	**A2a · MIN**	8, 6 → 6	β starts at 7	**Return 6**
5	**A2b · MIN**	First leaf = 4	α = 6; β = 4	**Prune remaining leaf 10**
6	**A2 / A**	A2 = 6; A = 6	Root α becomes 6	**A returns 6**
7	**B1a · MIN**	First leaf = 4	α = 6; β = 4	**Prune remaining leaf 8**
8	**B1b · MIN**	First leaf = 3	α = 6; β = 3	**Prune remaining leaf 9**
9	**B1 / B**	B1 = 4; B β = 4	α = 6 ≥ β = 4	**Prune all of B2 (4 leaves)**

WHAT HAPPENS AT THE ROOT?

1. **A returns 6.**
   MAX now has α = 6.

2. **B finds a reply worth 4.**
   At B, MIN has β = 4.

3. **Compare the bounds.**
   α ≥ β → 6 ≥ 4.

FINAL ROOT VALUE = 6

Minimax: 16 leaves · Alpha-Beta: 9

REAL LIFE

Pruning in Daily Life: Stop When You Cannot Win

You already prune. Alpha-Beta just makes it exact.

Job offers

You hold an offer of Rs 100,000 (that is α). Another company says its salary range ends at Rs 80,000 (that is β). α ≥ β, so you skip the interview.

Buying a phone

The first shop sells it for Rs 40,000 (α-style bound). At the next shop you learn the lowest price is Rs 45,000. No need to bargain: it cannot beat what you have.

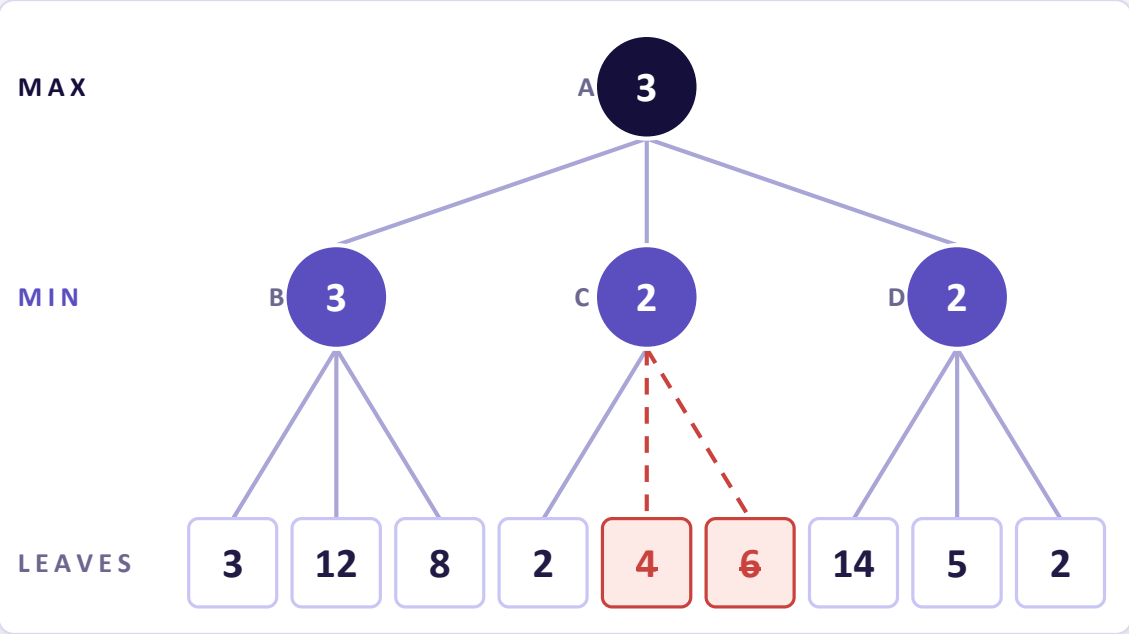
Cricket strategy

You find one bowling plan that already holds the batter to 20 runs. A rival plan that you quickly see allows 35 at least is not worth analysing further.

Pattern: you keep a “best so far” number. The moment a branch is guaranteed to be no better, you stop thinking about it. That is α ≥ β.

Alpha-Beta Trace Table (Exam Format)

Leaves are visited left to right. Start with α = −∞ and β = +∞.



Step	Node	Leaf	α	β	What happens
1	B	3	−∞	3	β = min(β, leaf)
2	B	12	−∞	3	β = min(β, leaf)
3	B	8	−∞	3	β = min(β, leaf)
4	C	2	3	2	β ≤ α → PRUNE 4 and 6
5	D	14	3	14	β = min(β, leaf)
6	D	5	3	5	β = min(β, leaf)
7	D	2	3	2	β = 2 ≤ α → D = 2 (last leaf)

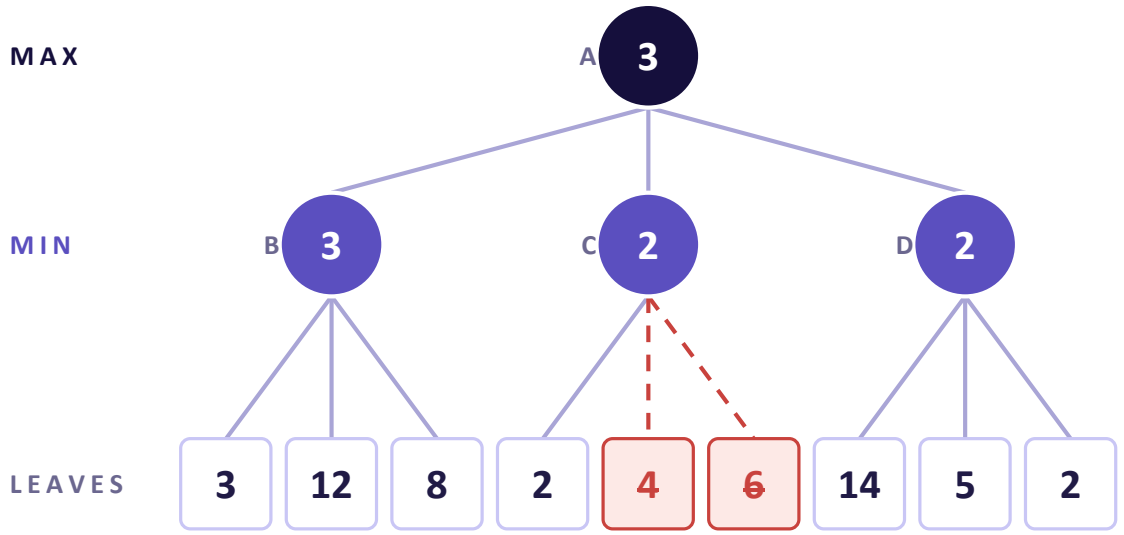
Result: A = 3 (move B). Same answer as Minimax, but only 7 of 9 leaves were looked at.

After B: B = 3, so α at the root becomes 3.
At C: first leaf 2 gives β = 2, which is ≤ α = 3. Prune 4 and 6.
At D: 14, then 5, then 2 → D = 2. Root stays 3.

Order Matters: Look at the Best Moves First

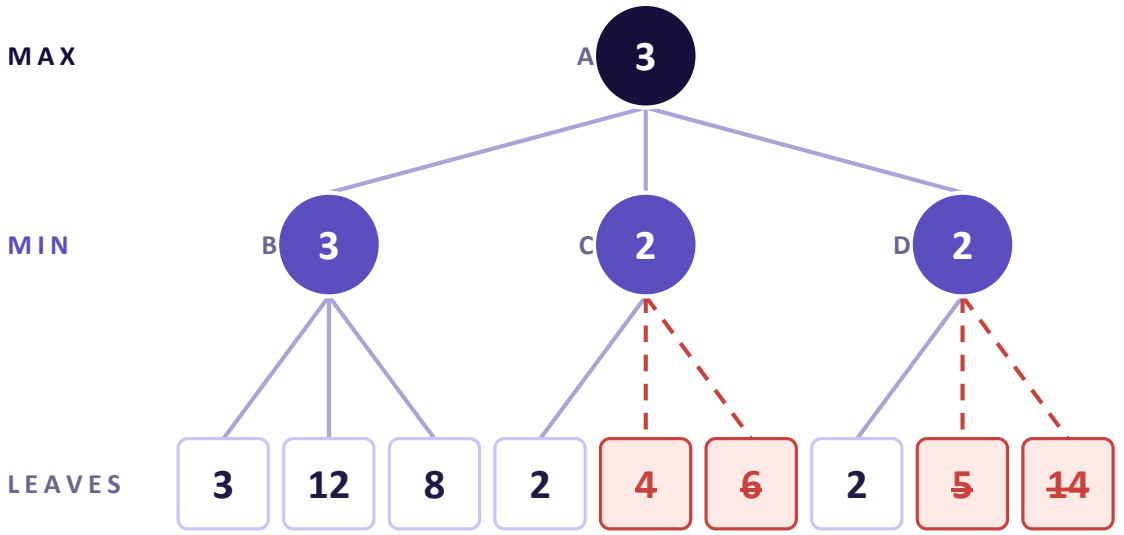
Same tree, same answer. Only the order of D's leaves changes.

ORIGINAL ORDER



Leaves checked: 7 · Pruned: 2

D REORDERED: 2, 5, 14



Leaves checked: 5 · Pruned: 4

Rule of thumb: with perfect ordering Alpha-Beta needs only about O(b^(m/2)) nodes, so it can search TWICE as deep in the same time. Random order gives roughly O(b^(3m/4)).

CODE · LINK TO THE LAB

Minimax and Alpha-Beta in Python

A tree is a nested list; a number is a leaf. Alpha-Beta = Minimax + two numbers + one "break".

MINIMAX

```
def minimax(node, is_max):
    if isinstance(node, int):
        return node
    vals = [minimax(c, not is_max)
            for c in node]
    if is_max:
        return max(vals)
    return min(vals)

tree = [[3,12,8],[2,4,6],[14,5,2]]
print(minimax(tree, True))  # 3
```

ALPHA-BETA

```
def alphabeta(node, is_max, a=-1e9, b=1e9):
    if isinstance(node, int):
        return node
    if is_max:
        best = -1e9
        for c in node:
            best = max(best, alphabeta(c, False, a, b))
            a = max(a, best)
            if a >= b: break        # PRUNE
        return best
    best = 1e9
    for c in node:
        best = min(best, alphabeta(c, True, a, b))
        b = min(b, best)
        if a >= b: break            # PRUNE
    return best
```

Lab task: add a counter for the leaves reached. You should get 9 for Minimax and 7 for Alpha-Beta, with the same answer 3.

Code written for this lecture; algorithms follow AIMA Sections 5.2–5.3.

How the Recursive Code Makes a Decision

The function keeps calling itself until it reaches a leaf score, then returns values upward.

1 INPUT THE TREE

```
[ [3, 12, 8],
  [2, 4, 6],
 [14, 5, 2] ]
```

Each inner list is a MIN node.

2 SOLVE THE MIN NODES

- B: min(3,12,8) = 3
- C: min(2,4,6) = 2
- D: min(14,5,2) = 2

3 RETURN TO MAX

MAX compares 3, 2, 2

max(3, 2, 2) = 3

MAX chooses branch B.

BASE CASE A leaf is a score: return it immediately.

RECURSIVE Evaluate children; MAX returns the largest, MIN the smallest. Alpha-Beta adds bounds and can stop early.

AVOID THESE TRAPS

Watch Out! Common Mistakes

Four things that cost marks in exams.

MIN picks the SMALLEST.

Do not let MIN “help” MAX. MIN is the opponent: they choose the lowest value, always.

Pruning never changes the answer.

If your Alpha-Beta result differs from Minimax, you have made a mistake. Check your α and β.

Values go UP, α and β go DOWN.

Minimax values are passed up from leaves to the root. α and β are passed down the path from the root.

Prune when α ≥ β, including equal.

If α equals β you can still prune. Also: visit leaves left to right unless told otherwise.

CHECK YOUR UNDERSTANDING · 6 QUESTIONS

Quick Quiz

Answer on paper or by raising your hand: A, B or C.

Q1 **In the price war, Shop A chooses between worst cases 10 and 25. Minimax picks…**

A) the option with 10 B) the option with 25 C) either

Q2 **A MIN node has children valued 8, 3 and 5. Its value is…**

A) 8 B) 3 C) 5

Q3 **True or False: Alpha-Beta Pruning can give a different best move from Minimax.**

A) True B) False

Q4 **What does α (alpha) store?**

A) The best score MAX can already guarantee B) The best score MIN can guarantee C) The number of leaves

Q5 **α = 5 and β = 4 at some node. What do we do?**

A) Keep searching B) Prune the remaining children C) Swap α and β

Q6 **Why can’t Minimax search the whole chess tree?**

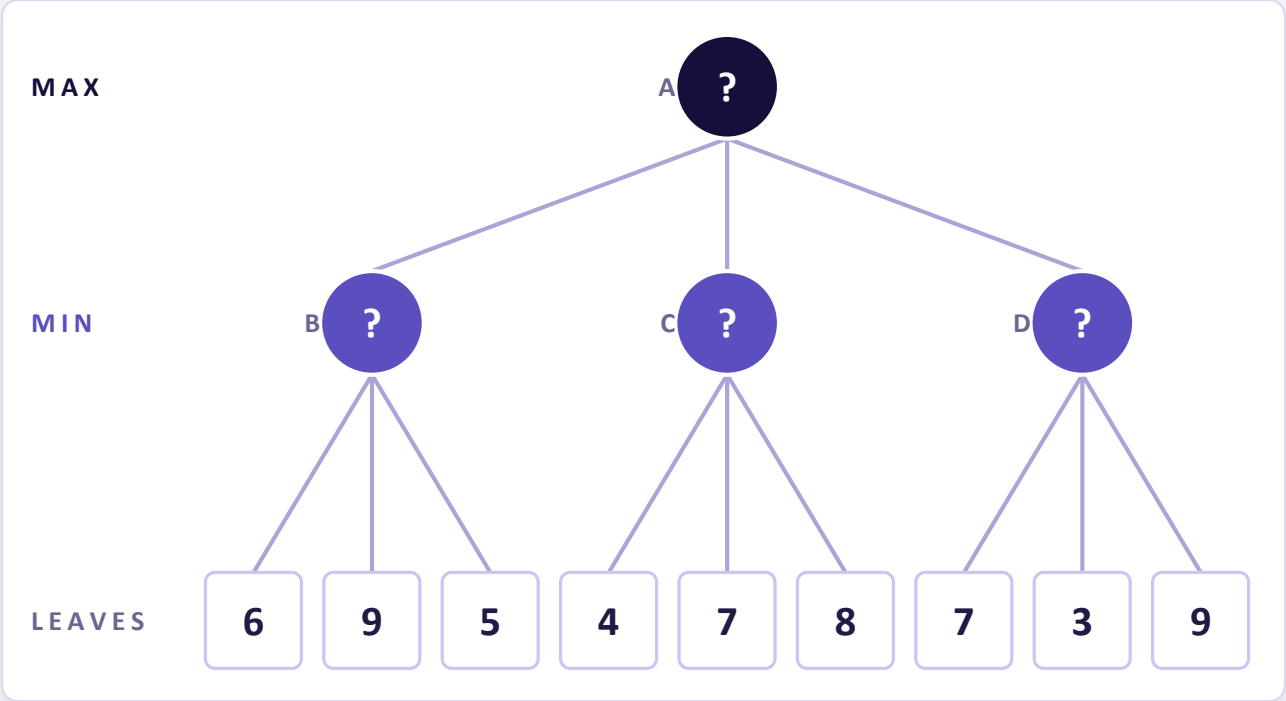
A) Far too many nodes ($\approx 10^{154}$) B) Chess has no winner C) Players are random

TRY IT YOURSELF · PAIRS · ~20 MIN · PAPER EXERCISE

Minimax and Alpha-Beta on a New Tree

MAX moves at A. MIN moves at B, C, D. Visit leaves left to right.

1. Minimax: find the values of B, C, D and A. Which move should MAX choose?
2. Alpha-Beta: write α and β at each step. Cross out every leaf that is pruned.
3. Count the leaves checked: Minimax vs. Alpha-Beta.
4. Check: is the Alpha-Beta answer the same as Minimax?
5. Bonus: reorder D's leaves to 3, 7, 9. How many leaves are pruned now?



	B	C	D	A
Minimax value				
Leaves checked (Alpha-Beta)				

WRAPPING UP

Key Takeaways

If you remember only six things from today:

1. In a two-player game we must plan for the opponent's BEST reply, not hope for a mistake.
2. Minimax: MAX takes the biggest value, MIN takes the smallest. Values are backed up from the leaves.
3. Minimax is optimal against a perfect opponent but looks at every node: O(b^m). Chess is far too big.
4. Alpha-Beta keeps α (MAX's guaranteed best) and β (MIN's guaranteed best). Prune when α ≥ β.
5. Alpha-Beta gives exactly the same answer as Minimax with fewer nodes. Good move ordering prunes more.
6. Real programs also use a depth limit and an evaluation function, an educated guess like a heuristic.

Russell & Norvig, Artificial Intelligence: A Modern Approach (3rd ed.), Chapter 5